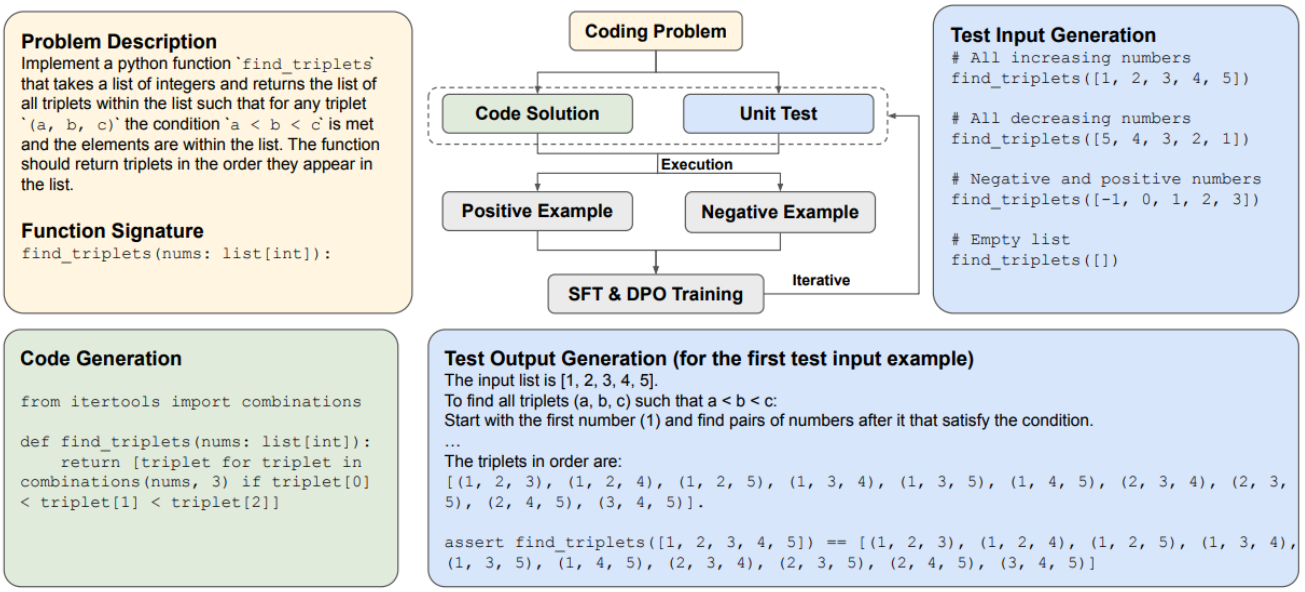
SOL-VER: A Self-Play Framework for Code and Test Co-Evolution

> #DPO #Finetuning #Self-evolve #Dataset @Adversarial



- What problems does this framework solve?
  - Shortage of high quality coding dataset. Self-generated coding datasets may lead to accumulation of mistakes-> degradation.
  - Bias in ability: good at coding, but poor at verification
  - This framework teaches the model to be both code generator and code verifier to have self-adversarial inference, just like in SPICE and self-challenge agent framework.
- What does this framework do?
  - Solver: given a problem, generate at least one correct answer C that can pass all checks in $\mathbf{T}$.
  - Verifier: given a problem, generate a testset that will fail all incorrect candidate answers and distinguish between the incorrect and correct.
  - Practically, these two characters can be played by the same LLM.
  - Scoring function:

    $$\text{Score}(C, \mathbf{T}) = \mathbb{E}_{T \sim \mathbf{T}}[\mathbb{I}(C(T) = \text{expected_output}(T))]$$

    The score of a candidate answer is the mean of the score over all tests in the testset. If the answer is correct, its score is 1 (passed everything).
  - Stage 1: SFT:
    - Sample some problems $\mathbf{P}$. For each $P \in \mathbf{P}$, generate a set of candidate answers. For each problem and each of its solutions, we also generate a set of test cases.
    - Define that:

      $$y = \begin{cases} 1 & \text{if } \text{Score}(C, \mathbf{T}) = 1 \text{ (i.e., passes all tests)} \\ 0 & \text{otherwise.} \end{cases}$$

    - We use the "good samples" (those with y = 1) to run SFT.

$$\mathcal{L}_{\text{SFT}_{\text{solver}}}(\theta) = -\mathbb{E}_{(P,C,\mathbf{T}):y=1}[\log S_\theta(C|P)]$$
$$\mathcal{L}_{\text{SFT}_{\text{verifier}}}(\phi) = -\mathbb{E}_{(P,C,\mathbf{T}):y=1}[\log V_\phi(\mathbf{T}|P,C)]$$

- Stage 2: DPO
  - We also want the model to distinguish between good answers and bad answers, not only just learning the pattern of the good answers.
  - For the solver, we use C^+-C^-pairs to optimize

    $$\mathcal{L}_{\text{DPO}_{\text{solver}}}(S_\theta^*; S_\theta) = -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{S_\theta^*(C^+|P)}{S_\theta(C^+|P)} - \beta\log\frac{S_\theta^*(C^-|P)}{S_\theta(C^-|P)}\right)\right]$$
  - For the verifier, the process is more complicated:
    - We define a good testcase as a testcase that at least one good candidate answer can pass (Score = 1).
    - We regard the testcase as a ground truth (positive sample). As we generate multiple testcases (major voting), we compare the ground truth with the rest of the testcases. If there exists one testcase such that it does not equal to the ground truth, it is a negative sample.
    - With this, run DPO.

      $$\mathcal{L}_{\text{DPO}_{\text{verifier}}}(V_\phi^*; V_\phi) = -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{V_\phi^*(\mathbf{T}^+|P,C)}{V_\phi(\mathbf{T}^+|P,C)} - \beta\log\frac{V_\phi^*(\mathbf{T}^-|P,C)}{V_\phi(\mathbf{T}^-|P,C)}\right)\right]$$

      As the testcases are generated via CoT, the CoT can also be finetuned.

- Result:
  - In the MBPP and LiveCodeBench benchmarks, code generation capability (Pass%) improved by an average of 19.63%, while test generation capability (accuracy) increased by an average of 17.49%.
  - Significant Leap in Verification Capability: As the number of iterations increases, the model's ability to generate accurate test cases has markedly improved, with a corresponding decrease in the false positive rate. This indicates that the model is becoming increasingly adept at “self-checking.”
  - No need for powerful giant models.
- Work Cited
  - Paper
  - Chat History